Programación de servicios y procesos

Tema 1: Programación multiproceso

Programas, ejecutables, procesos y servicios

- Un **programa** es una secuencia de instrucciones para realizar unas determinadas tareas.
- Un **ejecutable** es un programa en binario que puede interpretar directamente el ordenador.
- Un **proceso** es un programa en ejecución (o parte de él). El SSOO decide qué proceso entra y sale de la CPU.
- Un **servicio** es un programa en ejecución en segundo plano, que no interactúa con el usuario. Normalmente realizan tareas para el SSOO.

Procesos

Elementos de un proceso

Se compone de:

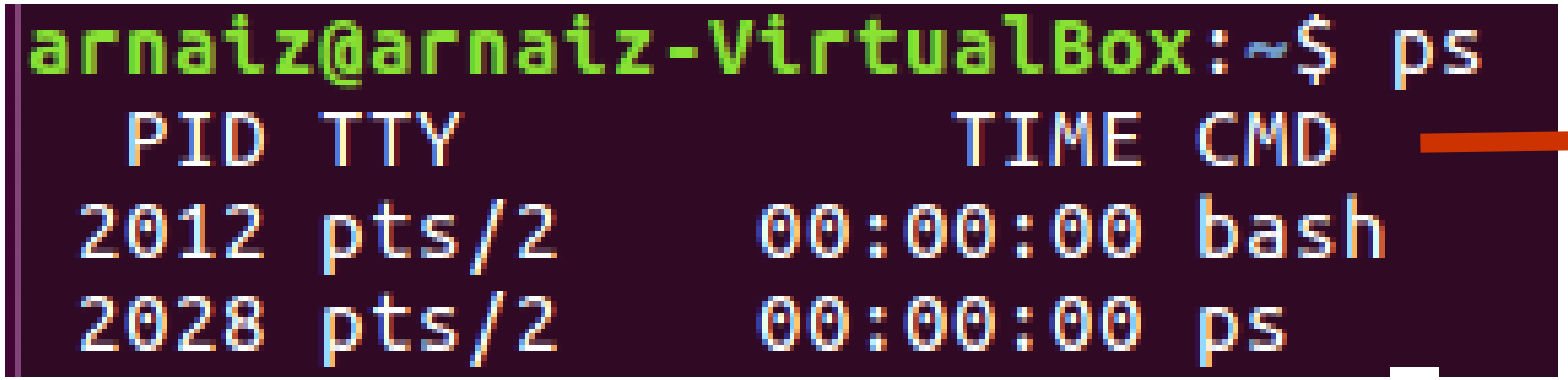
- Código ejecutable
- Datos
- Pila del programa
- Contador de programa (puntero de instrucciones)
- Puntero de pila y otros registros
- Información necesaria para ejecutar el programa

Bloque de Control de Proceso (BCP)

Cuando un proceso sale de la CPU temporalmente, al volver a entrar debe retomarse en el mismo estado en que se encontraba. La información necesaria para ello se guarda en el **Bloque de Control de Proceso**, que se compone de:

- Identificador del proceso (único para cada proceso)
- Estado del proceso
- Contador de programa
- Registros de la CPU
- Información de planificación de CPU (prioridad entre otros)
- Información de gestión de memoria
- Información contable (cantidad de tiempo de CPU, tiempo real consumido…)
- Información de estado de E/S (dispositivos, archivos abiertos…)

ps (process status)



Nombre del proceso

Identificador del proceso

Terminal asociado L/E

Tiempo en CPU

ps -f

```
arnaiz@arnaiz-VirtualBox:~$ ps -f
UID          PID    PPID  C STIME TTY          TIME CMD
arnaiz      2012    2005  0 20:12 pts/2    00:00:00 bash
arnaiz      2384    2012  0 20:14 pts/2    00:00:00 ps -f
```



Usuario

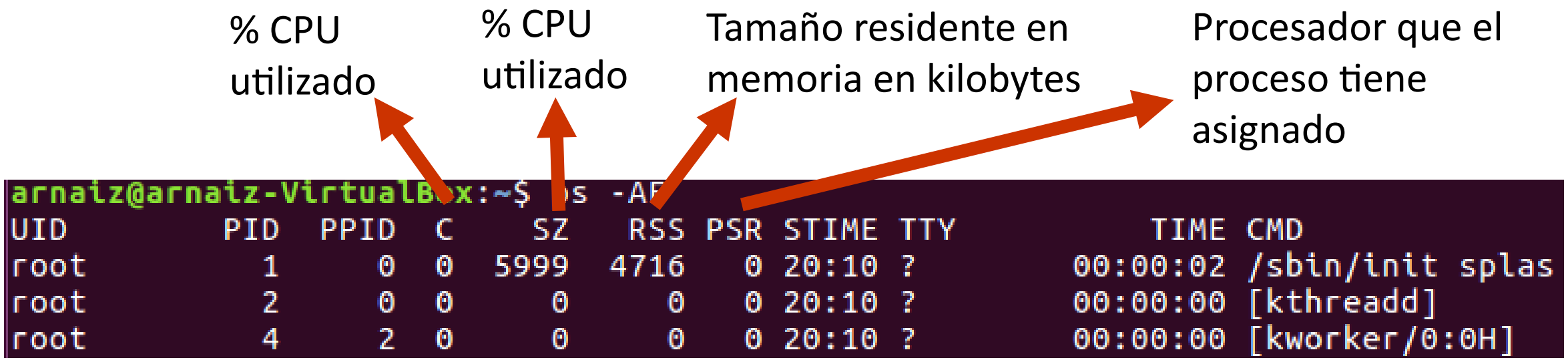


Identificador del proceso padre



Hora de inicio del proceso

ps -AF


% CPU utilizado
% CPU utilizado
Tamaño residente en memoria en kilobytes
Procesador que el proceso tiene asignado
arnaiz@arnaiz-VirtualBox:~$ ps -AF
UID PID PPID C SZ RSS PSR STIME TTY TIME CMD
root 1 0 0 5999 4716 0 20:10 ? 00:00:02 /sbin/init splas
root 2 0 0 0 0 0 20:10 ? 00:00:00 [kthreadd]
root 4 2 0 0 0 0 20:10 ? 00:00:00 [kworker/0:0H]

Procesos en Linux



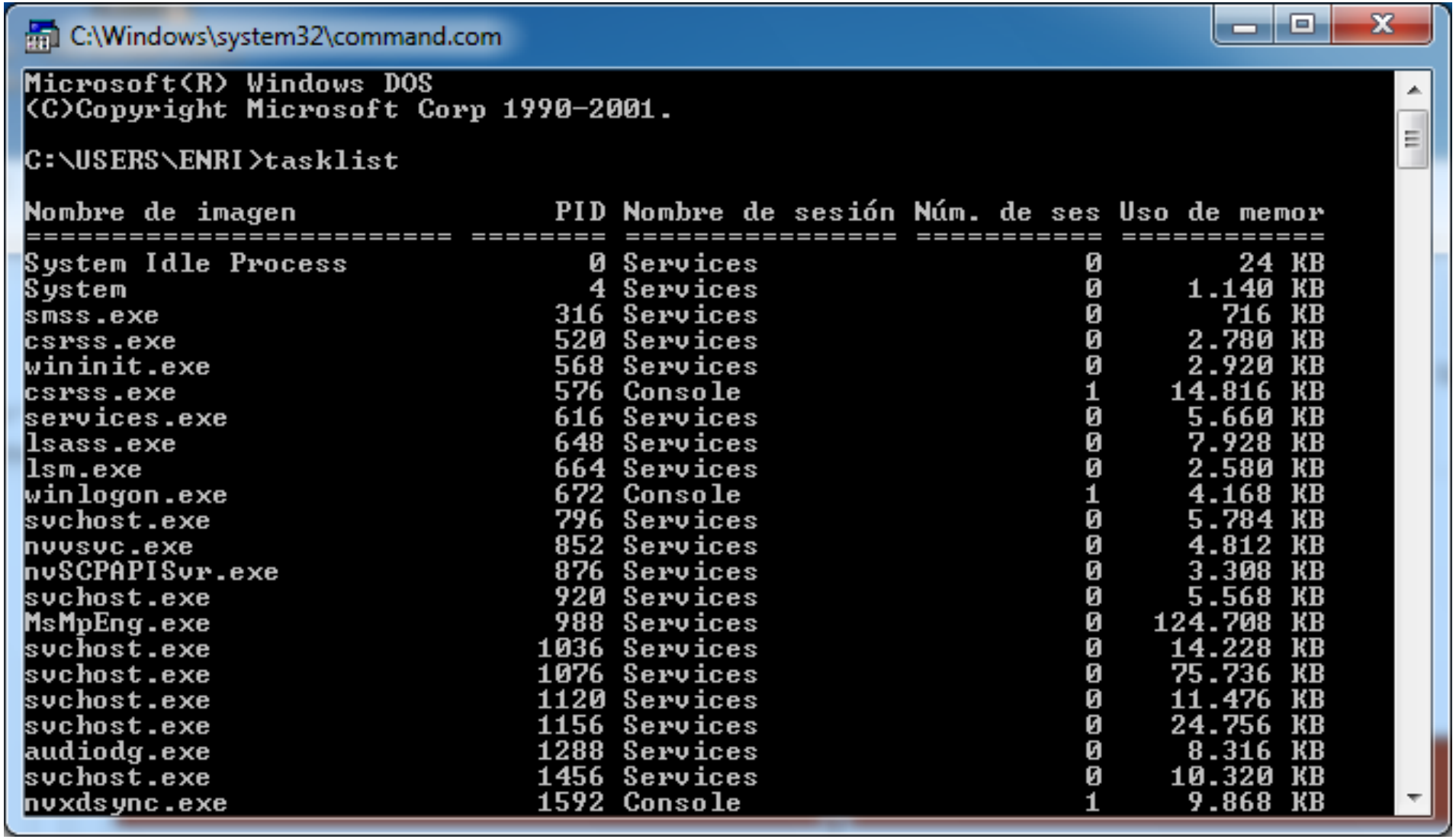
Monitor del sistema

Procesos | Recursos | Sistemas de archivos

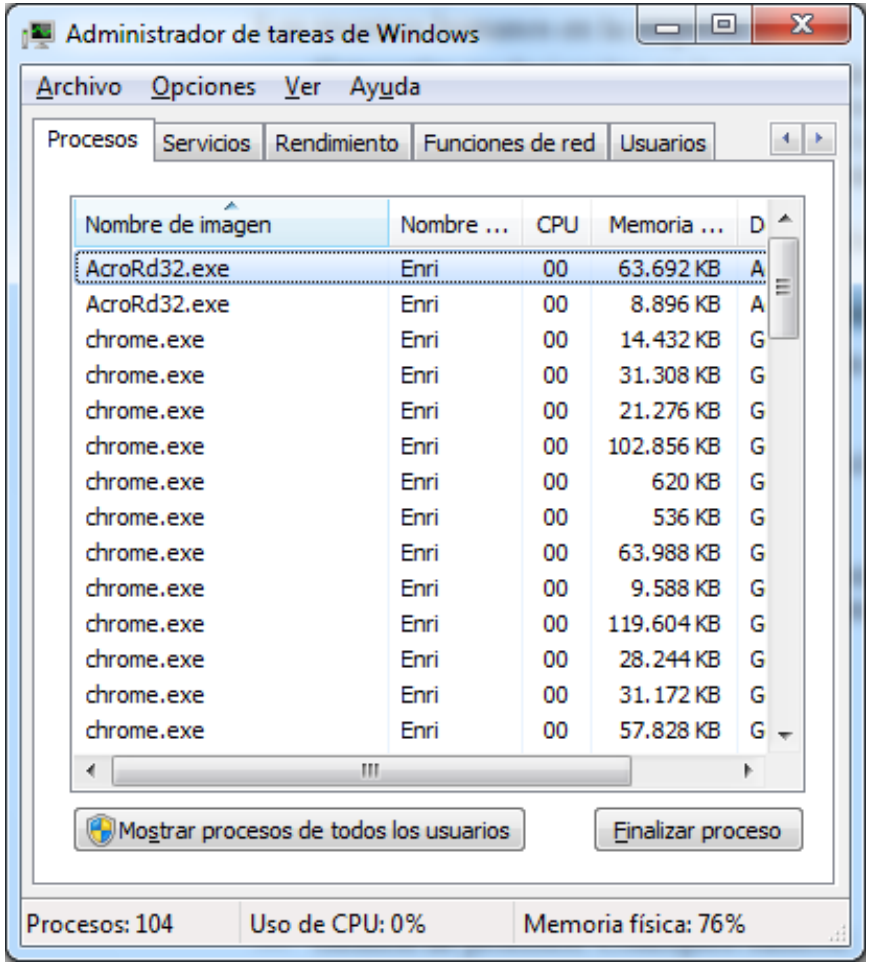
Nombre del proceso	Usuario	% CPU	ID	Memoria	Prioridad
at-spi2-registryd	arnaiz	0	1406	392,0 KiB	Normal
at-spi-bus-launcher	arnaiz	0	1392	544,0 KiB	Normal
bamfdaemon	arnaiz	0	1334	6,9 MiB	Normal
compiz	arnaiz	1	1631	128,8 MiB	Normal
dbus-daemon	arnaiz	0	1125	1,1 MiB	Normal
dbus-daemon	arnaiz	0	1400	396,0 KiB	Normal
dconf-service	arnaiz	0	1581	464,0 KiB	Normal
deja-dup-monitor	arnaiz	0	2059	668,0 KiB	Normal
evolution-addressbook-factor	arnaiz	0	1743	1,8 MiB	Normal
evolution-addressbook-factor	arnaiz	0	1787	1,6 MiB	Normal
evolution-calendar-factory	arnaiz	0	1668	24,9 MiB	Normal
evolution-calendar-factory-sul	arnaiz	0	1719	23,6 MiB	Normal
evolution-calendar-factory-sul	arnaiz	0	1730	23,5 MiB	Normal
evolution-source-registry	arnaiz	0	1617	2,5 MiB	Normal
gnome-keyring-daemon	arnaiz	0	1216	488,0 KiB	Normal
gnome-session-binary	arnaiz	0	1463	1,5 MiB	Normal
gnome-software	arnaiz	0	1660	36,6 MiB	Normal
gnome-system-monitor	arnaiz	2	2782	9,7 MiB	Normal
gpg-agent	arnaiz	0	1425	176,0 KiB	Normal
gvfs-afc-volume-monitor	arnaiz	0	1784	616,0 KiB	Normal

Procesos en Windows

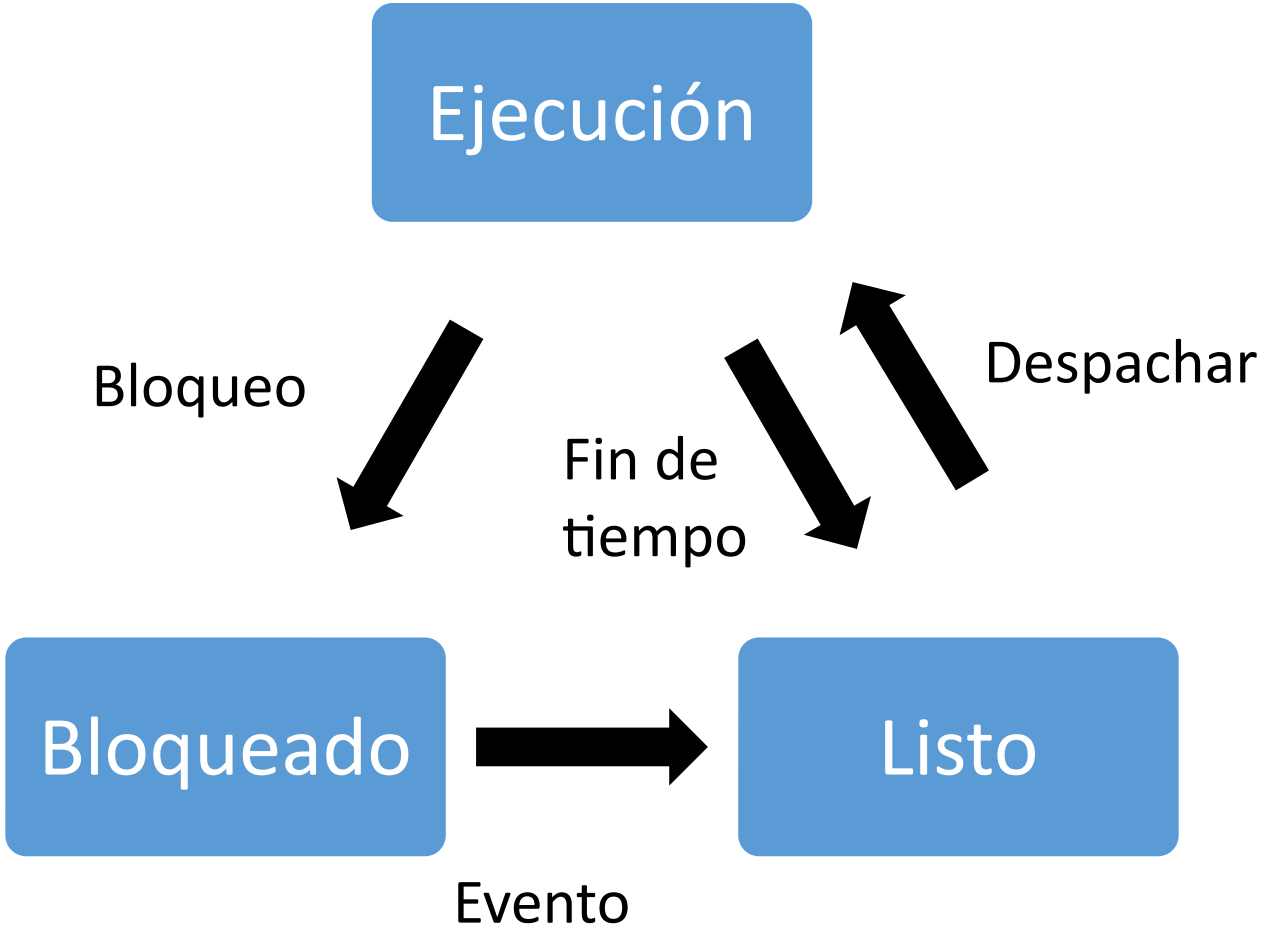
tasklist



Administrador de tareas de Windows [CTRL+Alt+Supr]



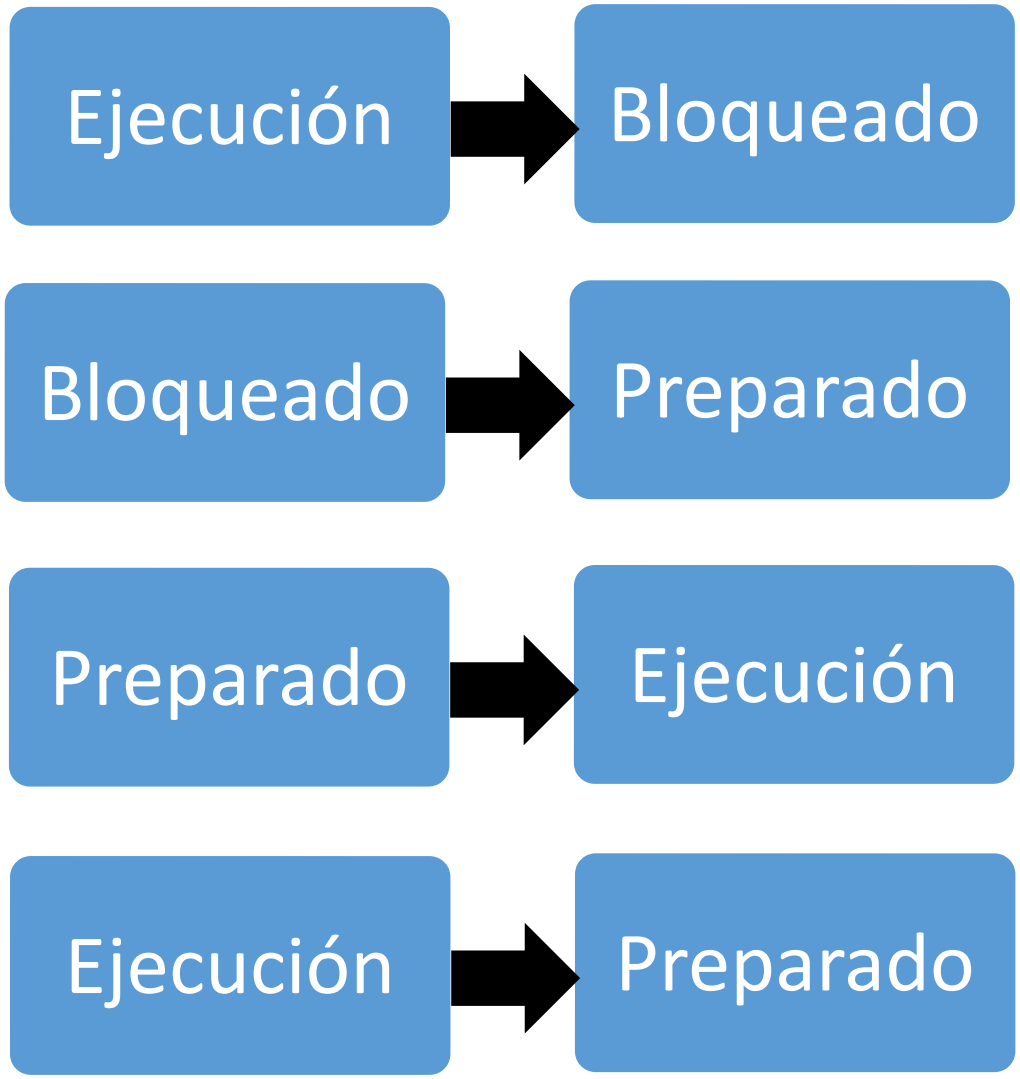
Estados de un proceso



Estados de un proceso

Ejecución	El proceso está haciendo uso de la CPU
Bloqueado	El proceso está esperando a que ocurra un evento (por ejemplo que termine una operación de E/S)
Preparado	El proceso está listo para ejecutarse cuando se le de paso

Transiciones entre procesos



Planificación de procesos

Niveles de planificación de procesos

- Planificación a **largo plazo**: Se realiza el control de admisión de procesos a ejecutar. Se utiliza mucho en *batch*.

- Planificación a **medio plazo**: Se selecciona qué procesos se añaden o retiran de memoria principal.

- Planificación a **corto plazo** (*dispatcher*): Se selecciona el siguiente proceso a ejecutar.

Tipos de planificación de procesos

- **No apropiativa**: El proceso en ejecución sigue usando la CPU mientras quiera.

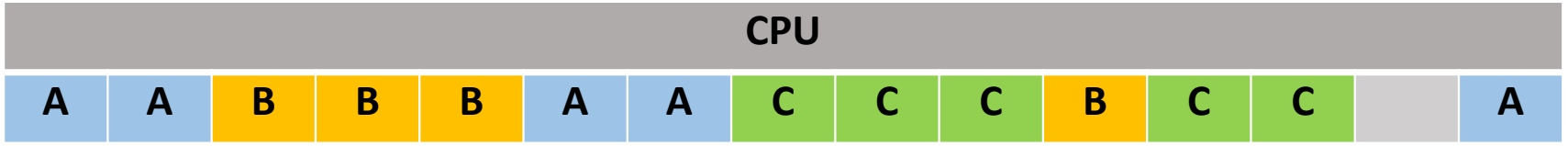
- **Apropiativa**: El SSOO puede expulsar a los procesos de la CPU. Esta política supone mayor coste que la no apropiativa, pero da un mejor servicio al conjunto de procesos. Evita la monopolización de la CPU.

Medidas de la planificación

- **Uso de la CPU**: Porcentaje de tiempo que la CPU está en uso. Se debe maximizar.
- **Rendimiento**: Número de trabajos terminados por unidad de tiempo. Se debe maximizar.
- **Tiempo de retorno**: Tiempo transcurrido entre la llegada de un proceso y su finalización. Se debe minimizar.
- **Tiempo de servicio**: Tiempo dedicado a tareas productivas.
- **Tiempo de espera**: Tiempo que el proceso está esperando.

Ejemplo de cálculo de medidas de la planificación

A, B y C llegan en el momento 0.



Uso de CPU A: (5*100)/15 = 33,333%
Uso de CPU B: (4*100)/15 = 26,666%
Uso de CPU C: (5*100)/15 = 33,333%
Uso de CPU: (14*100)/15 = 93,33%

Rendimiento A: 1/15 = 0,066
Rendimiento B: 1/11 = 0,090
Rendimiento C: 1/13 = 0,076
Media: 0,077

T. de retorno A: 15
T. de retorno B: 11
T. de retorno C: 13
Media: 13

T. de servicio A: 5
T. de servicio B: 4
T. de servicio C: 5
Media: 4,666

T. de espera A: 10
T. de espera B: 7
T. de espera C: 8
Media: 8,333

Algunos ejemplos de algoritmos de planificación

- **FCFS** (First Come, First Served): El primero en llegar es el primero en entrar a la CPU. No apropiativo.

  + Fácil de implementar.

  - Tiempo de espera promedio bastante largo.

- **SJF** (Shortest Job First): Primero el trabajo más corto. No apropiativo.

  + Minimiza el tiempo de espera medio.

  - Riesgo de inanición de los procesos largos.

  - En la práctica se basa en estimaciones de la duración de los procesos.

Algunos ejemplos de algoritmos de planificación

- **Por prioridad**: Entra el de mayor prioridad. Apropiativo o no apropiativo.

  - Los procesos con prioridad más baja tienen riesgo de inanición. Solución: envejecimiento (aumentar prioridad con el tiempo).

- **RR** (Round Robin): Como *FCFS* pero cada proceso dispone de un tiempo máximo (*Q)*. Apropiativo.
  - Si *Q* es muy grande, los procesos terminan de usar la CPU antes del límite de tiempo. En este caso es idéntico a *FCFS*.
  - Si *Q* es muy pequeño, al estar cambiando constantemente de proceso, el rendimiento disminuye mucho.

Ejercicio: Planificación de procesos

Los siguientes procesos llegan en el orden indicado en la u.t. 0:

A: duración 8 u.t.

B: duración 3 u.t.

C: duración 2 u.t.

A) Dibujar la ejecución de los procesos en la CPU con planificación FCFS y RR con ventanas de CPU de 3 u.t., en condiciones ideales: la CPU está 100% dedicada a estos procesos, el tiempo para cambiar de proceso es despreciable y todo el tiempo empleado por los servicios es productivo.

B) Calcular para cada una de las planificaciones: uso de la CPU de cada proceso y total, rendimiento de cada proceso y rendimiento medio, tiempo de retorno de cada proceso y tiempo de retorno medio, tiempo de servicio de cada proceso y tiempo de servicio medio, y tiempo de espera de cada proceso y tiempo de espera medio.

Actividades: Planificación de procesos [PSP_T1_Planificación]

A) Dibujar la ejecución de los procesos siguientes con FCFS, SJF, Prioridades apropiativo y no apropiativo, RR con ventana de 2 u.t.), teniendo la CPU dedicada al 100% a los procesos, sin tener en cuenta el tiempo para cambiar de proceso, y siendo todo el tiempo empleado por los procesos productivo:

Proceso	Llegada*	Duración**	Prioridad***
A	1	2	5
B	1	2	1
C	6	3	3
D	7	5	2

* Aunque A y B llegan en la misma unidad de tiempo, A llega antes que B.
** A igualdad de duración, se prioriza el que haya llegado primero.
*** La prioridad máxima es la 1 y la mínima la 10.

Actividades: Planificación de procesos [PSP_T1_Planificación]

B) A partir de la planificación del apartado anterior, calcular los siguientes datos para SJF y RR:

Uso de CPU A:
Uso de CPU B:
Uso de CPU C:
Uso de CPU D:
Uso de CPU total:

Rendimiento A:
Rendimiento B:
Rendimiento C:
Rendimiento D:
Media de rendimiento:

Tiempo de retorno A:
Tiempo de retorno B:
Tiempo de retorno C:
Tiempo de retorno D:
Media de tiempo de retorno:

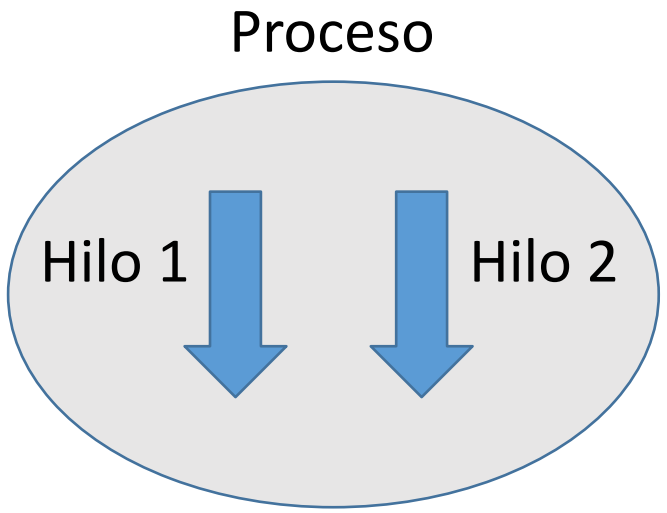
Tiempo de servicio A:
Tiempo de servicio B:
Tiempo de servicio C:
Tiempo de servicio D:
Media de tiempo de servicio:

Tiempo de espera A:
Tiempo de espera B:
Tiempo de espera C:
Tiempo de espera D:
Media de tiempo de espera:

Hilos

Hilos

Un hilo (*thread*) es una secuencia de código en ejecución dentro de un proceso. Un hilo no puede ejecutarse por sí mismo, necesita la supervisión de un proceso padre.



Los hilos se emplean en programas que necesitan realizar varias tareas simultáneamente. Por ejemplo un procesador de texto puede tener un hilo que se encargue de imprimir un documento y otro que atienda las peticiones del usuario.

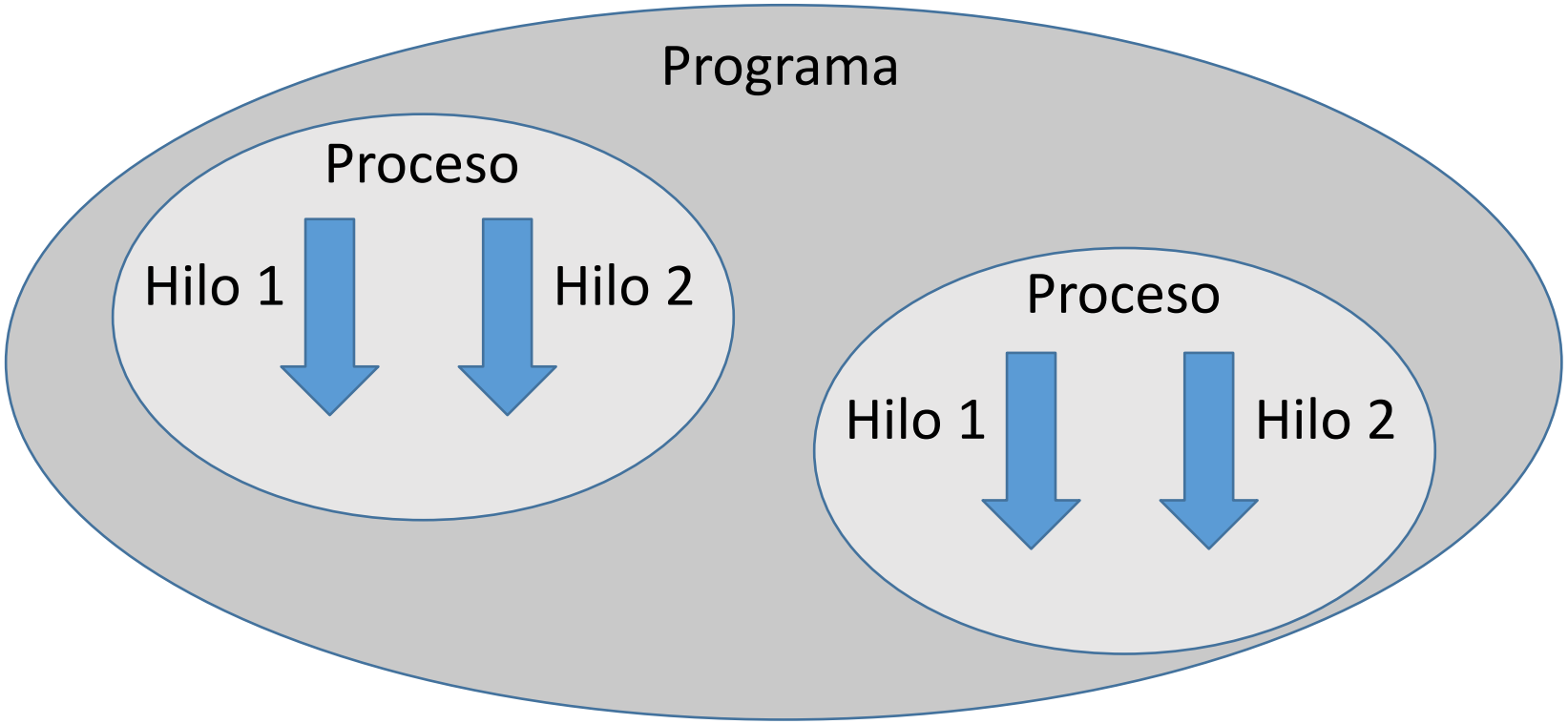
Hilos vs Procesos

Hilos	Procesos
Comparten el espacio de memoria del usuario.	Poseen espacios de memoria independientes.
Los cambios de estado son menos costosos.	Los cambios de estado son más costosos.

Programación concurrente

Programación concurrente

Un programa se puede componer de varios procesos. Cuando las instrucciones de estos procesos se solapan intercaladamente, se dice que son procesos concurrentes.



Beneficios de la programación concurrente

- **Mejor aprovechamiento de la CPU**: Cuando un proceso está esperando un evento, otro puede aprovechar la CPU.
- **Velocidad de ejecución**: Al dividir el programa en procesos, éstos se pueden repartir mejor entre el o los procesadores.
- **Solucionan problemas de naturaleza concurrente**:
  - **Sistemas de control**: Capturan datos, los analizan y actúan.
  - Tecnologías **web**: Los servidores atienden múltiples peticiones.
  - Aplicaciones basadas en **GUI**: Mientras se atiende al usuario, la aplicación puede estar realizando tareas.
  - **Simulaciones**: Modelización de sistemas físicos con autonomía.
  - **S**istemas **G**estores de **B**ases de **D**atos: Atienden múltiples peticiones.

Problemas de la programación concurrente

- **Exclusión mutua**: Cuando dos o más procesos hacen uso a la vez de una variable se puede producir inconsistencia de datos (uno puede estar escribiendo en ella y otro leyéndola). Para conseguir la exclusión mutua se define la **región crítica**, que limita el acceso a la variable a un único proceso a la vez.

- **Condición de sincronización**: A veces se producen situaciones en que un proceso tiene que esperar a que otro llegue a cierto punto. Para ello se proporcionan mecanismos para bloquear procesos a la espera de que ocurra el evento necesario.

Condiciones de Bernstein

- **Conjunto de lectura**: Formado por las variables a las que se accede en modo lectura.

- **Conjunto de escritura**: Formado por las variables a las que se accede en modo escritura.

Condiciones de Bernstein

Para que dos conjuntos se puedan ejecutar concurrentemente se deben cumples 3 condiciones:

1. La intersección entre las variables leídas de un conjunto de instrucciones y las escritas de otro, debe ser vacío.

$$L(Ii) \cap E(Ij) = \emptyset$$

2. La intersección entre las variables escritas de un conjunto de instrucciones y las leídas de otro, debe ser vacío.

$$E(Ii) \cap L(Ij) = \emptyset$$

3. La intersección entre las variables escritas de un conjunto de instrucciones y las escritas de otro, debe ser vacío.

$$E(Ii) \cap E(Ij) = \emptyset$$

Ejemplo de condiciones de Bernstein

Instrucciones	
I1	x = y + 1
I2	y = x + 2
I3	z = a + b

Conjunto de lectura	Conjunto de escritura
L(I1)={y}	E(I1)={x}
L(I2)={x}	E(I2)={y}
L(I3)={a,b}	E(I3)={z}

Conjunto I1 e I2

Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$
Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$
Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$

Conjunto I2 e I3

Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$
Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$
Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$

Conjunto I1 e I3

Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$
Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$
Conjunto I1 e I2	Conjunto I2 e I3	Conjunto I1 e I3
$L(I1) \cap E(I2) = \{y\}$	$L(I2) \cap E(I3) = \emptyset$	$L(I1) \cap E(I3) = \emptyset$
$E(I1) \cap L(I2) = \{x\}$	$E(I2) \cap L(I3) = \emptyset$	$E(I1) \cap L(I3) = \emptyset$
$E(I1) \cap E(I2) = \emptyset$	$E(I2) \cap E(I3) = \emptyset$	$E(I1) \cap E(I3) = \emptyset$

Ejemplo de implementación de hilos concurrentes [PSP_T1_Concurrencia]

```
public class HiloSimple extends Thread { //Hereda de Thread

    public void run() { //Se sobrecarga el método run
        for (int i=0; i<10; i++)
            System.out.println("Dentro del hilo.");
    }
}

public class UsaHilo {

    public static void main(String[] args) {

        HiloSimple hs = new HiloSimple(); //Declara el hilo
        hs.start(); //Lanza el hilo
        for (int i=0; i<15; i++)
            System.out.println("Fuera del hilo.");
    }
}
```

Ejemplo de implementación de hilos concurrentes [PSP_T1_Concurrencia]

```
public class HiloSimple ex
    public void run() { /
        for (int i=0; i<1
            System.out.pr
    }
}

public class UsaHilo {

    public static void ma

        HiloSimple hs = n
        hs.start(); //Lan
        for (int i=0; i<1
            System.out.pr
    }
}
```

```
Fuera del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Dentro del hilo.
Fuera del hilo.
Fuera del hilo.
Fuera
```

Realiza la actividad **PSP_T1_Thread**



Programación paralela

Programación paralela

Los programas paralelos están diseñados para ejecutarse en sistemas con varios procesadores. El programa se divide en procesos que se ejecutan en procesadores distintos.

Tradicionalmente la programación paralela se ha utilizado en centros de supercomputación para resolver problemas complejos en el menor tiempo posible. Actualmente con la aparición de los procesadores de múltiples núcleos, su uso se ha incrementado.

Los procesos cooperan con un fin común, para lo cual necesitan intercambiar información.

Modelos de programación paralela

En función de cómo se realiza el intercambio de información, tenemos dos modelos de programación paralela:

- Modelo de **memoria compartida**: Los procesadores comparten la memoria físicamente. Un valor escrito en memoria por un procesador puede ser leído por otro.

- Modelo de **paso de mensajes**: Cada procesador dispone de su propio espacio de memoria, solo accesible por él. El procesador que necesita la información de otro se la solicita, y éste se la envía.

Ventajas e inconvenientes de la programación paralela

☺ Ventajas

- Proporciona ejecución simultánea de tareas.
- Disminuye el tiempo total de ejecución de una aplicación.
- Resolución de problemas complejos y de grandes dimensiones.
- Utilización de recursos en red.
- Disminución de costos.

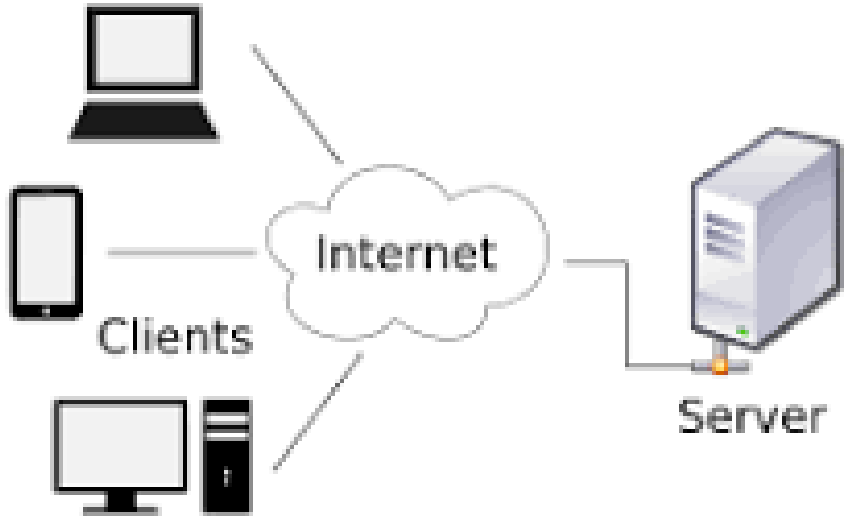
☹ Inconvenientes

- Los compiladores y entornos de programación para sistemas paralelos son más complejos de desarrollar.
- Los programas paralelos son más complicados de realizar.
- El consumo de energía es mayor al haber varios elementos implicados.
- Mayor complejidad en el acceso a datos.
- La comunicación y sincronización entre procesos.

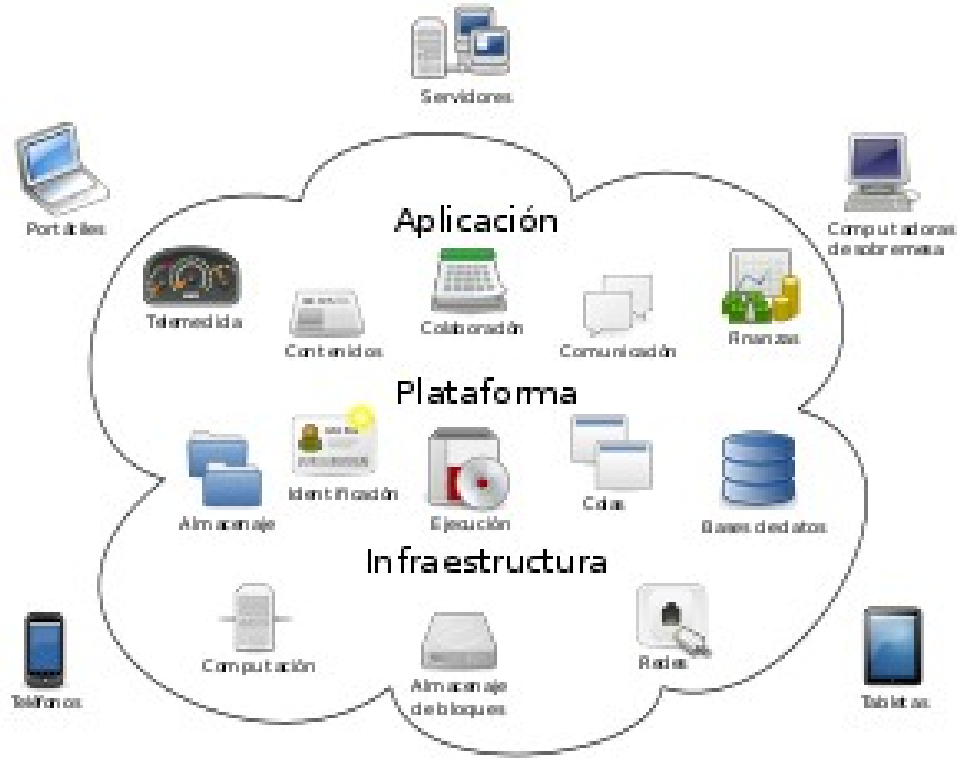
Programación distribuida

Programación distribuida

Un sistema distribuido se basa en compartir recursos. Algunos ejemplos son *World Wide Web* y la computación en la nube.



Arquitectura cliente-servidor (wikipedia)



Computación en la nube (wikipedia)

Modelos de programación distribuida

En función de cómo se comunican sus procesos, existen 3 modelos de sistemas distribuidos:

- **Sockets**: Proporcionan los puntos para la comunicación entre procesos. Es la base de la comunicación actual.
- Llamada de procedimientos remotos o **RPC** (*Remote Procedure Call*): Permite a un programa cliente llamar a un procedimiento de otro que se está ejecutando en un proceso servidor. El proceso servidor define en su interfaz de servicio los procedimientos que pueden ser utilizados.
- Invocación remota de objetos o **RMI** (*Remote Method Invocation*): Objetos distribuidos. Un objeto que reside en un proceso puede invocar a métodos de un objeto que reside en otro proceso distinto.

Ventajas e inconvenientes de la programación distribuida

☺ Ventajas

- Se pueden compartir recursos y datos.
- Capacidad de crecimiento incremental.
- Mayor flexibilidad (se distribuye la carga de trabajo entre distintos ordenadores).
- Alta disponibilidad, tolerantes a fallos.
- Soporte de aplicaciones inherentemente distribuidas.
- Carácter abierto y heterogéneo.

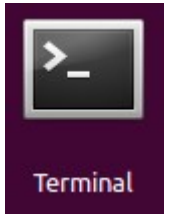
☹ Inconvenientes

- Aumento de la complejidad.
- Problemas en la comunicación (pérdida de mensajes, saturación…).
- Problemas de seguridad (ataques de denegación de servicio…).

Creación de procesos

Creación de programas C en Linux

Linux permite la creación de procesos mediante programación en C.
Los pasos para crear un programa en Linux en C son:

1. Ejecutar Terminal. 

2. Ejecutar editor: `arnaiz@arnaiz-VirtualBox:~$ gedit`
3. Realizar programa y guardar con extensión “.c”.
4. Crear el ejecutable: `arnaiz@arnaiz-VirtualBox:~$ gcc programa.c -o programa`
5. Ejecutarlo: `arnaiz@arnaiz-VirtualBox:~$ ./programa`

Creación de procesos

Para crear un proceso existe la función *fork*:

```
#include <unistd.h>
pid_t fork(void);
```

Al llamarla se crea un proceso hijo que es una copia exacta en código y datos del proceso que lo crea (padre), salvo el PID y la memoria que ocupa.

Las variables del proceso hijo son una copia de las del padre, por lo que al modificarlas en uno de los procesos no se refleja en el otro.

La función puede devolver:

- -1: Si se produce un error.
- 0: Si no devuelve error y nos encontramos en el proceso hijo.
- PID del hijo: Si no se produce error y nos encontramos en el proceso padre.

Creación de procesos

Para obtener el identificador del proceso actual:

```
pid_t getpid (void);
```

Para obtener el identificador del proceso padre:

```
pid_t getppid (void);
```

Ejemplo de creación de procesos [psp_t1_fork.c]

```c
#include <stdlib.h>
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

void main() {

  pid_t pid, wpid;

  pid = fork(); //se crea el proceso hijo
  if (pid == -1) //error
  {
    printf("Error al crear proceso hijo.");
    exit(-1);
  }
  if (pid ==0) //proceso hijo
  {
    printf("Soy el proceso hijo, mi PID es %d y el de mi padre %d.\n", getpid(), getppid());
  }
  else //proceso padre
  {
    wpid = wait(NULL); //esperamos a que termine el hijo para no terminar el programa antes
    printf("Soy el proceso padre, mi PID es %d y el de mi padre %d.\n", getpid(), getppid());
  }
  exit(0);
}
```

Ejemplo de creación de procesos [psp_t1_fork.c]

```
arnaiz@arnaiz-VirtualBox:~$ ./psp_t1_fork
Soy el proceso hijo, mi PID es 3798 y el de mi padre 3797.
Soy el proceso padre, mi PID es 3797 y el de mi padre 2022.
```

```
arnaiz@arnaiz-VirtualBox:~$ ps
  PID TTY          TIME CMD
 2022 pts/0    00:00:00 bash
 3090 pts/0    00:00:00 ps
```

Shell de Ubuntu

Creación de procesos en JAVA

Java puede crear procesos pesados y ejecutar tareas o aplicaciones como por ejemplo PAINT o notepad.

Se puede crear con ProcessBuilder y con Runtime.

A continuación vemos un ejemplo con ProcessBuilder y otro con Runtime.

Creación de procesos en JAVA

```
public class LanzadorProcesos {
public void ejecutar(String ruta){
ProcessBuilder pb;
try {
pb = new ProcessBuilder(ruta);
pb.start();
} catch (Exception e) {
// TODO Auto-generated catch block
e.printStackTrace();
}
}
/**
* @param args
*/
public static void main(String[] args) {
String ruta=
"C:\\Program Files (x86)\\Adobe\\Reader 11.0\\Reader\\AcroRd32.exe";
LanzadorProcesos lp = new LanzadorProcesos();
lp.ejecutar(ruta);
System.out.println("Finalizado");
}
}
```

Creación de procesos en JAVA

Con Runtime.

```
public class ProcessDemo {

        public static void main(String[] args) throws Exception {
                Runtime r = Runtime.getRuntime();
                Process p = r.exec("firefox");
                p.waitFor(10, TimeUnit.SECONDS);
                p.destroy();
        }
}
```

NOTA: Para destruir un proceso podemos utilizar el método destroy();

Codifica un proceso mediante Runtime y otro mediante ProcessBuilder, una vez creado destrúyelos.
Se adjunta las cabeceras

import java.IOException;
import java.util.Arrays;
A) Con Runtime.
B) Con ProcessBuilder.
C) Destruir procesos.
D) Comenta como lo has hecho

